Data-file generation for structural-analysis software

Luiz Eduardo Lollato Junqueira

Supervisor: Prof. Dr. Heitor Miranda Bottura

Universidade Estadual Paulista (UNESP), Campus de Bauru, Faculdade de Engenharia de Bauru, Departamento de Engenharia Civil, Área: Estruturas — December 2001

> English translation of the graduation thesis *Geração de arquivos de dados para software de análise estrutural* (Portuguese original: `junqueira-2001-geracao-arquivos-dados-software-analise-estrutural.pdf` in this folder). Translated by the author in 2026 with AI assistance; the text follows the original closely, including its 2001 statements, except where a bracketed *[2026 note]* says otherwise. The annexes of the original (DLEARN variable list, PPD source listing, example input and output) are not reproduced here because the repository holds them in better form — see the end of this document.

Contents

1. Introduction and objectives
2. Literature review — 2.1 The finite element method · 2.2 The DLEARN software · 2.3 The Turbo Pascal compiler
3. Methodology
4. Example of use of PPD
5. Results obtained
6. Analysis of results and conclusions
7. Bibliography
8. Annexes — where to find them in this repository

Abstract

The purpose of this work was to speed up the creation of the data file for structures to be analysed with the DLEARN software, by writing a computer program in the Pascal language that generates such a file.

One characteristic of this program — from here on called **PPD** (the initials of *Pré Processador Dlearn*) — is that its algorithm was built so that it can easily incorporate updates of DLEARN; to that end, comments were placed throughout the source program indicating the subroutines (procedures) used, which are listed in the order in which the program executes them. In addition, the consistency of the variables is checked against lower and/or upper limits, avoiding user errors. Whenever possible, the interdependence that may exist between variables is also checked.

Other features implemented in the program were: data supplied as answers to questions, and on-screen display of the name of the source-program variable corresponding to the datum being supplied.

The examples solved, such as the one presented in chapter 4, confirm how easy DLEARN becomes to use when PPD is used to create the data file of the structures.

1. Introduction and objectives

The project developed in this graduation work is attached to the supervisor's field of activity, structural analysis by the finite element method. It concerns the application of a new algorithm for the solution of the equation of motion to a software for the dynamic analysis of structures with multiple degrees of freedom.

Its development allowed the student to go deeper into structural theory as well as into the aspects and applications of the finite element method, including contact with the solution of dynamic problems, which is not covered in the undergraduate courses.

The research project used the DLEARN software, which performs the dynamic analysis of structures by the finite element method and whose source program is available in HUGHES (1987); besides treating the subject of structural dynamic analysis, that book also describes the concepts of the algorithms used in assembling the subroutines that make up the system, which allows the development of research on new alternative algorithms and supports research already developed by the supervisor.

The DLEARN system is written in the FORTRAN language, which, although very convenient for calculation — it has been used by researchers for decades — offers a certain rigidity and lack of convenience in supplying the problem data: a text file must be created, composed only of numbers arranged in pre-established lines and columns. This is notoriously a laborious, uninteresting and unproductive step.

The purpose of the present project is to speed up the creation of the data file for the structures to be analysed by DLEARN, by writing a computer program in the PASCAL language to generate that file, giving the system greater speed and safety of use.

2. Literature review

2.1 The finite element method

DLEARN uses the formulation of the Finite Element Method (FEM), one of the most widely used methods for the analysis of three-dimensional structures. The great majority of commercial programs for this kind of analysis use it. The method rests on a basic principle of discretisation, which may be defined as a procedure in which a complex problem of large extent is divided — discretised — into small equivalent elements or components. After discretisation a model is used to describe the behaviour of each component, and afterwards compatibility is established between all the

elements of the continuum. This compatibility between the various parts is generally represented by the displacements.

Since its proposal in 1954 (ARGYRIS, 1954; TURNER, CLOUGH, MARTIN, TOPP, 1956) the finite element method has received the dedication of a growing number of researchers in several areas of engineering and applied mathematics.

Initially the subject of papers in traditional journals, the method rapidly reached such prominence that the first textbooks devoted to it soon appeared (CLOUGH, 1960, 1965; ZIENKIEWICZ, 1967; MIKHLIN, 1966). The growing application of the method to the most varied fields soon made some of these texts incomplete, requiring enlarged re-editions such as that of ZIENKIEWICZ in 1971, and caused the appearance of new periodicals almost exclusively devoted to disseminating new ideas and applications on the subject (*International Journal for Numerical Methods in Engineering*, Wiley-Interscience).

With this rapid evolution of the method, groups of researchers produced an enormous variety of finite elements. Each has a number of points, and at each point a series of related variables (parameters), whether displacement or rotation. Over each element an approximating function (linear, quadratic, cubic, etc.) is used, relating the nodal values of each parameter.

Several works published in Brazil have used the finite element method for structural analysis, such as BARBOSA (1978), who compared the continuous-medium technique with the discrete treatment. PRUDENTE (1983) analysed structures made of bracing panels, formed by beams and columns rigidly connected to each other, and individual columns not subject to flexural-torsional effects. The structural system considered was a three-dimensional structure formed by vertical panels and individual columns braced horizontally by the slabs. The static analysis was carried out by the displacement method.

BECKER (1989) added structural cores, based on the theory of VLASSOV (1962), to the study of the three-dimensional interaction between the various structural elements. The structural cores (sets of wall-columns or of columns and beams for stairs or lifts) are responsible for absorbing the forces due to lateral wind load. MORI (1992) and MATIAS (1997), building on BECKER's program, carried out the geometrically non-linear analysis of three-dimensional structures of tall buildings considering the resisting cores.

RIOS (1991) analysed multi-storey building structures without considering the formation of panels and cores, treating them as beams and columns connected to the slab. That work took into account the eccentricities between the elements, but without considering the torsional stiffness of beams and columns. The displacement method was used, combined with matrix structural analysis techniques.

BRUNELI (1987) and BALCAZAR (1991) analysed several multi-storey building structures subject to wind action, now considering the flexural stiffness of the slabs — that is, the plate behaviour of the slabs, which until then had been treated as diaphragms infinitely rigid in their horizontal plane and with negligible transverse

stiffness. For this the authors employed the finite element method through the displacement method. BALCAZAR, to better implement the computer program developed, used series and parallel sub-structuring techniques. However, these two works apply only to buildings with rectangular plans.

The finite element method solves the equations of structural dynamics by replacing the differential equations of the problem by systems of simultaneous linear equations.

The dynamic analysis of a structure whose description of motion under a given excitation requires knowledge of the displacement in more than one coordinate of some chosen system constitutes a multi-degree-of-freedom system. The matrix equilibrium equation in those coordinates has the form

```
[M]{ẍ} + [C]{ẋ} + {q(t)} = {P(t)}
```

where the vector {x} is the vector of displacements in the adopted coordinates. The mass matrix is [M], damping is represented by [C], the internal forces by the term {q} and {P(t)} is the vector of excitations.

Solving the equation above gives the displacements in the chosen coordinates, arranged in the vector {x}, for the instant considered.

The internal forces caused by relative displacements of the various parts of the structure imply strength requirements, and the internal stresses associated with them, if periodic, may cause fatigue in the component materials of the structure. Knowledge of them (in value and frequency) may therefore also be a design datum.

The excitations P(t) and the initial conditions are the causes of the displacements.

When the internal forces are considered, with the displacement known and linear behaviour assumed (a well-described problem), the equation above takes the form

```
[M]{ẍ} + [C]{ẋ} + [K]{x} = {P(t)}
```

Any damping present in the motion is associated with the matrix [C], the values of whose elements are obtained from an assumed hypothesis for the behaviour of the structure. Damping is quite difficult to measure, and it is considered indirectly through coefficients in [C] that seek to reproduce, with convenient fidelity and simplicity, the physical phenomenon that occurs.

A software for the analysis of multi-degree-of-freedom structures by the finite element method does nothing more than solve the matrix equation above, giving as results the displacements and the quantities that depend on them, such as velocities, accelerations and internal forces.

2.2 The DLEARN software

The source code of DLEARN, a structural-analysis software by the finite element method, as well as the instructions for creating the data files needed to use it, are in

HUGHES (1987). It has elements for the analysis of three-dimensional trusses and two-dimensional elements for the analysis of plane stress, plane strain and axisymmetric solids without torsion. The version made available on the Internet had three-dimensional brick elements added.

DLEARN uses the equation above to solve problems, and its structure has the following sequence:

```
(a) DATA INPUT  →  (b) SOLUTION BY FEM  →  (c) OUTPUT OF RESULTS
─────────────── the step this work intends to improve
```

Figure 1 — Solution sequence of the DLEARN software.

Each of the three main stages is described below.

a — Data input. DLEARN's data input consists of inserting the data of the structure to be analysed into a text file, positioning the values in lines and columns pre-established in the user manual available in HUGHES (1987). This type of data input is inherited from punched cards, which were the only data-input systems available at the beginning of DLEARN's development; this is why the lines are limited to 80 characters. This type of input is typical of software that uses FORTRAN as its programming language; SAP version 90 may be cited as another example.

An example of the data file for a two-dimensional structure is shown in Figure 2 *[a listing of a DLEARN card file in the original; see* `examples/trelica/trelica-zace.dat` *in this repository]*.

Because of this characteristic of positioning values in pre-established lines and columns, data input is an uninteresting and laborious step, which makes the use of DLEARN for the most varied purposes difficult; this part is the focus of this graduation work.

The modification of the data input was made by creating a separate program, written and compiled in PASCAL, the PPD, built so that it can easily incorporate updates of DLEARN; to that end comments were placed throughout the source program indicating the subroutines (procedures) used, listed in the order of execution. In addition, the consistency of the variables is checked against lower and/or upper limits, avoiding user errors. Whenever possible the interdependence that may exist between variables is also checked.

Other features implemented in the program were: data supplied as answers to questions, and on-screen display of the name of the source-program variable corresponding to the datum being supplied.

All the DLEARN variables used by PPD are presented on pages 1 to 16 of the annexes *[2026 note: superseded by* `docs/dlearn-input-format.md`*, which was derived from the Fortran and corrects several column positions]*.

b — Solution by the finite element method. The solution of structures by the finite element method consists of the basic principle of discretisation described above. After

discretisation a model is used to describe the behaviour of each component, and afterwards compatibility is established between all the elements of the continuum, generally represented by the displacements and their derivatives.

b.1 — Assembly of the mass and stiffness matrices and of the global action vectors. There are two options for this step. In the original configuration, the system assembles the effective mass matrix with a view to Crout factorisation. For the solution via Hermitian algorithms, the global stiffness and mass matrices were assembled separately. The latter is then inverted to obtain $[M]^{-1}[K]$. Next, depending on the order of the member of the family one wishes to use, a corresponding routine must be used to assemble the sub-matrices and then the global matrices, building the matrices that define the system of equations to be solved.

b.2 — Solution of the system of equations. At this stage the two analysis paths proceed in parallel and independently. In one of them the system-solution routines were used in full. In the other, for the step-by-step advance of the motion, there is an independent routine for solving the system of equations. The symmetry property, exhaustively exploited to optimise the solution of structural systems, is not present.

b.3 — Obtaining the acceleration. The solution via Hermitian algorithms, however, gives as answers for the advance of the motion the displacements and velocities. Once these are obtained, the corresponding acceleration can be computed by direct application of the equilibrium equation, which therefore always gives consistent values — that is, values that correspond to a situation of exact equilibrium (with no residual external forces) for the applied actions and the numerical responses at a given instant.

b.4 — Reports with the results of the analysis. Once the values obtained are rearranged to conform to the software in its original configuration, the available resources for printing result reports can be used, which makes any comparisons one wishes to make easier. Thus the procedure at this stage is unique, regardless of the solution method employed.

After programming the routines and the modifications to the original code needed to carry out the steps described above, problems available in the literature were analysed, which allowed the program to be debugged and the first conclusions on the use of the new methods in computer systems to be drawn.

A comment on the performance of the Hermitian algorithms in question is appropriate. In fact, one parameter that is of interest in the performance analysis of a method of this kind is the processing time. For that, it is more convenient to use a pre-existing program, so that differences in programming efficiency are purged from the comparison. On the other hand, given the differences between the methods, there is no full parallelism between the operations performed, which makes comparisons difficult even in such cases. For example, a specifically developed software would not need to rearrange the results to print reports. However, at the present stage, where linear elastic problems of not very large size are concerned, such an analysis was hampered by the short duration of the processing involved and by the fact that the comparison software was built for high efficiency in exactly that situation. On the other hand, when problems involving non-linearity are evaluated (to which, as everything so far indicates,

the single-step Hermitian algorithm should lend itself very well), a specific analysis must be carried out if a reliable comparative measure is wanted.

c — Output of results. The results are output by writing a text file with all the information obtained, as with any structural-analysis software, visual or not.

2.3 The Turbo Pascal compiler

For the development of the program in PASCAL, WOOD (1987), FRANCHIN (1998) and RINALD (1993) were used as reference sources; the last presents a brief history of the language.

The PASCAL programming language was created as an educational tool at the beginning of the 1970s by Prof. Niklaus Wirth of the Technical University in Zürich. Its creator named it PASCAL in honour of the great mathematician Blaise Pascal, inventor of one of the first known logical machines. It was based on some structured languages then in existence, ALGOL and PL/I, trying to make it as easy as possible to learn.

PASCAL only gained popularity when it was adopted by the University of California, San Diego, in 1973. In the same period, in its courses, implementations for minicomputers and microcomputers were also made.

In its first implementations it was not very friendly to the programmer, since several steps were needed to obtain an executable program: first the program had to be written in a text editor, then compiled, linked and assembled.

Despite all the initial difficulties, its educational purpose and its ease of programming, PASCAL began to be used by programmers of other languages, becoming — to Niklaus' own surprise — a commercial product. However, it was only at the end of 1983 that the American software house Borland International launched TURBO PASCAL for microcomputers. In that same period there was great growth in the microcomputer market. The Turbo Pascal compiler was one of the best-selling compiler series of all time and made the language popular, particularly on the PC platform, thanks to its balance between simplicity and power.

TURBO PASCAL manages to put in a single environment a text editor much like WordStar and SideKick, source code linked to an error-detection system, and a single-pass compiler, which makes it very fast.

TURBO PASCAL in its first versions, up to version 3, had an initial screen that also served as the main menu; all existing routines were in a single file and the executable program had a .COM structure.

From version 4 on, TURBO had some conceptual changes, the main ones being of environment, making it friendlier, like most modern software. The auxiliary routines were divided according to the activity they perform in the program, and the program structure itself became .EXE, which by its characteristics separates the various areas of the program into 64-kbyte segments, besides new real types.

From version 5.0 on, ways of debugging programs in the DOS environment were also added. An object-oriented language structure was added too, allowing the creation of records containing programming structures.

The version used was 7.0, which presents some improvements over version 5.0; it was provided by the Faculdade de Engenharia de Bauru and is available on the microcomputers installed in the Faculty's Computer Laboratory.

Among the advantages of using Turbo Pascal 7.0 for this work, the following stand out:

- It is a language taught in the disciplines *Introduction to Computer Science* and *Numerical Methods*, given in the 3rd and 4th semesters, respectively, of the civil engineering course of the Faculdade de Engenharia de Bauru.
- On compilation, the executable program reaches a size much smaller than that of any other known visual language.
- It is considerably easy to update by any other undergraduate student.

3. Methodology

The methodology employed to reach the proposed objective can be described by the activities listed below, in order:

1. Initially, a literature review of dynamic analysis was carried out so that the student would become familiar with the terminology used and the basic variables and concepts involved in the problem. Dynamic analysis is not part of any regular discipline of the civil engineering course, hence the need for this parallel study.
2. The structure of the data file was studied in the literature and then an algorithm was developed containing the steps needed to speed up data input into DLEARN.
3. The activity following the study of the algorithm was the programming of the algorithm in the PASCAL language; the sources consulted were RINALD (1993) and the lecture notes of FRANCHIN (1997), who taught the course *Introduction to Computer Science* in the second semester of 1997.
4. Once the software was finished, there was a phase of tests and debugging for any adjustments.
5. Finally this descriptive report of the work carried out was written.

4. Example of use of PPD

An example of the use of PPD will be developed for a truss subject to dynamic loads, shown in the figure below.

It is a truss with the dimensions and geometry shown in Figure 3, under the action of a load at the node, direction and sense indicated in the sketch, of 1000 units, constant from the beginning of the motion. All bars have a cross-sectional area of 10 units, the modulus of elasticity is 2.1×10^5 and the mass density is 7.3×10^{-6}, all these values being consistent with the system of units adopted. There is no physical damping. The period of the first mode of vibration (T) is close to 0.23 seconds.

```
   P = 1000 ↓ at node 3
 1 ──────── 2 ──────── 3
 │  \     │  \     │      height 600
 4 ──────── 5 ──────── 6
   800      800           (total 1600)
```

Figure 3 — Example truss.

The purpose of this example is to show how easily the data file can be created with PPD. The step-by-step use of PPD for the example above is described below.

- Run PPD
- Choose the option: CREATE FILES

```
Type a name for the file (with extension): Trelica.txt

Line 00 – Data echo
  Echo data file (Y/N): Y
Line 01 – Title of the structure
  Title for the structure: UM EXEMPLO DINAMICO DE TRELICA COM VARIOS NOS
Line 02 – General data of the structure
  Execution code (0/1): 1
  Which type of analysis (0/1/2): 0
  Read the restart file (Y/N): N
  Write to the restart file (Y/N): N
  Print node and element data (Y/N): Y
  Rank-check code (0/1/2): 0
  Number of time sequences: 1
  Number of nodal histories to be printed: 1
  Number of space dimensions: 3
  Total number of nodes of the structure: 6
  Number of degrees of freedom per node (2 to 6): 3
  Number of load cases: 1
  Number of load-time functions: 1
  Number of points defining the load-time functions: 1
  Number of element groups: 1
  Data of lines 0, 1 and 2 OK (Y/N)?: Y
Line 03 – Time-sequence data
  Time sequence number: 1 (filled automatically by the software)
  Number of steps of the sequence: 100
  Every how many steps are disp/vel/acc printed: 1
  Every how many steps are stress/strain printed: 1
  Every how many steps is the information above plotted: 1
  Number of iterations of the corrector loop: 1
  Parameter Alpha: -0.10
  Parameter Beta: 0.3025
  Parameter Gamma: 0.60
  Step Δt: 0.023
  Data OK (Y/N)?: Y
Line 04 – Nodal-history data
  Number of the node to be printed or plotted: 3
  Degree of freedom of the node: 2
  Kinematic quantity specifier: 1
  Data OK (Y/N)?: Y
Line 5.1 – Nodal coordinates
  Node number: 1
```

```
  Generation type: 2
  X coordinate: 0.00
  Y coordinate: 600.00
  Z coordinate: 0.00
Line 5.2 – Data for generation of nodal coordinates
  X coordinate: 1600.00
  Y coordinate: 600.00
  Z coordinate: 0.00
  Data of the last 1 generation point(s) OK (Y/N)?: Y
Line 5.3 – Nodal increment
  Number of additional nodes in direction 1: 2
  Node-numbering increment in direction 1: 1
  Data OK (Y/N)?: Y
  Finished entering nodes (Y/N)?: N
Line 5.1 – Nodal coordinates
  Node number: 4
  Generation type: 2
  X coordinate: 0.00
  Y coordinate: 0.00
  Z coordinate: 0.00
Line 5.2 – Data for generation of nodal coordinates
  X coordinate: 1600.00
  Y coordinate: 0.00
  Z coordinate: 0.00
  Data of the last 1 generation point(s) OK (Y/N)?: Y
Line 5.3 – Nodal increment
  Number of additional nodes in direction 1: 2
  Node-numbering increment in direction 1: 1
  Data OK (Y/N)?: Y
  Finished entering nodes (Y/N)?: Y
Line 06 – Boundary-condition data
  Number of the first node of the sequence: 1
  Node-number increment for generation: 1
  Number of the last node of the sequence: 6
  Boundary code of degree of freedom 1: 0
  Boundary code of degree of freedom 2: 0
  Boundary code of degree of freedom 3: 1
  Data OK (Y/N)?: Y
  Finished entering boundary conditions (Y/N)?: N
Line 06 – Boundary-condition data
  Number of the first node of the sequence: 1
  Node-number increment for generation: 0
  Boundary code of degree of freedom 1: 1
  Boundary code of degree of freedom 2: 1
  Boundary code of degree of freedom 3: 1
  Data OK (Y/N)?: Y
  Finished entering boundary conditions (Y/N)?: N
Line 06 – Boundary-condition data
  Number of the first node of the sequence: 4
  Node-number increment for generation: 0
  Boundary code of degree of freedom 1: 1
  Boundary code of degree of freedom 2: 1
  Boundary code of degree of freedom 3: 1
  Data OK (Y/N)?: Y
  Finished entering boundary conditions (Y/N)?: Y
Line 7.1 – Prescribed nodal forces / imposed kinematic conditions
  Node number: 3
  Generation type: 0
```

```
  Prescribed force / imposed condition of degree of freedom 1: 0
  Prescribed force / imposed condition of degree of freedom 2: -1000
  Prescribed force / imposed condition of degree of freedom 3: 0
  Data OK (Y/N)?: Y
  Finished entering forces / imposed kinematic conditions (Y/N)?: Y
Line 08 – Time functions
  Instant: 0.00
  Value of the time function: 1.00
  Data OK (Y/N)?: Y
Line 09 – Initial kinematic conditions
  Is there any initial kinematic condition (Y/N)?: N
Line 10 – Element types
  Which element type is present in this group (1/2)?: 2
Line 10.2.1 – BAR element group
  Number of elements in this group: 8
  Number of materials in this group: 1
  Maximum number of nodes for all elements of this group (2/3): 2
  Number of stress/strain histories: 0
  Print stress output (Y/N): Y
  Number of the existing load function for body forces: 0
  Number of integration points (1/2/3): 1
  Mass-matrix code (0/1/2): 1
  Implicit/explicit code (0/1): 0
Line 10.2.2 – Material properties
  Modulus of elasticity: 210000.0
  Density: 0.000073
  Rayleigh damping coefficient on the mass matrix: 0.00
  Rayleigh damping coefficient on the stiffness matrix: 0.00
  Area: 0.00
  Data OK (Y/N)?: Y
Line 10.2.4.1 – Element definition
  Element number: 1
  Material number: 1
  First node: 1
  Second node: 2
  Generation code: 1
  Data OK (Y/N)?: Y
Line 10.2.4.2 – Element definition
  Number of additional elements in direction 1: 2
  Element-number increment for direction 1: 1
  Node increment for direction 1: 1
  Data OK (Y/N)?: Y
  Finished entering the element data of this group (Y/N)?: N
  [... the same for element 3 (nodes 4–5), element 5 (nodes 2–4, increment 2) and
   element 6 (nodes 2–5, increment 2) ...]
  Finished entering the element data of this group (Y/N)?: Y
```

The data are written to a text file, and this file can be "read" and interpreted by DLEARN. Figure 4 of the original shows the generated file *[2026 note: the file as generated in 2001 is listed on p. 76 of the PDF; the corrected version produced by the Python PPD is `examples/trelica/trelica-hermitian.dat`].*

With the input data file one can run DLEARN and type the name of the file written by PPD (`Trelica.txt`) for DLEARN to generate the output file, which in this case was named `Trelica.sai` (Figure 5). Page 76 of the annexes presents this input file, and the output file generated by DLEARN is presented in the annexes from page 77 on.

5. Results obtained

The result obtained was a computer program in PASCAL whose execution generates a data file for running the DLEARN software; this program is on the CD at the end of the work.

Besides the PPD software, this CD contains Turbo Pascal 7.0, the PPD source file, and this final report of the graduation work. The opening screen of PPD is shown in Figure 6.

Although not part of the initial proposal of this graduation work, for the purpose of distributing and improving PPD a website was built in HTML at `http://www.geocities.com/dlearnbr` (redirector `http://dlearnbr.cjb.net`). This website contains all the information needed to run PPD, as well as an electronic copy of this graduation work, besides the already compiled software ready for use. The site also presents several links on the subject treated here, and makes available for download the compiled DLEARN and its FORTRAN source code, in case any lecturer or student becomes interested in the subject. *[2026 note: the site no longer exists; this repository replaces it.]*

6. Analysis of results and conclusions

From the program obtained, one may conclude that there is greater speed, agility and safety in its use for generating the data files of the structures.

The objective of making the generation of data files easier — allowing several examples to be created quickly, in an open-source software suited to research, without the need for a heavy pre-processing system as in commercial systems — is considered to have been satisfactorily fulfilled.

The source code of the program is presented in the annex between pages 17 and 75.

7. Bibliography

1. ARGYRIS, J.H. *Energy Theorems and Structural Analysis.* Aircraft Eng. – Butterworth, 1954.
2. BALCAZAR, E.A.S.G. *Análise linear de estruturas tridimensionais pelo método dos elementos finitos utilizando subestruturas.* 1991. Dissertação (Mestrado), EESC, Universidade de São Paulo, São Carlos.
3. BARBOSA, J.A. *Edifícios com paredes de seção aberta contraventadas por lintéis, sob carga lateral.* 1978. Dissertação (Mestrado), EESC-USP, São Carlos.
4. BECKER, E.P. *Edifícios altos: interação tridimensional das peças de contraventamento.* 1989. Dissertação (Mestrado), EESC-USP, São Carlos.
5. BOTTURA, H.M. *Uma família de algoritmos hermitianos para a integração direta das equações de dinâmica das estruturas.* 1997. Tese (Doutorado), EESC-USP, São Carlos.

6. FRANCHIN, M.N. *Notas de aula – Introdução à Ciência da Computação.* Faculdade de Engenharia de Bauru, 1997.
7. HUGHES, T.J.R. *The Finite Element Method.* Prentice-Hall, Englewood Cliffs.
8. MATIAS JR., I.G. *Análise não-linear de estruturas tridimensionais de edifícios altos com núcleos resistentes sobre fundações flexíveis.* 1997. Dissertação (Mestrado), EESC-USP.
9. MORI, D.D. *Os núcleos estruturais e a não-linearidade geométrica na análise de estruturas tridimensionais de edifícios altos.* 1983. Tese (Doutorado), EESC-USP.
10. PRUDENTE, M. *Análise de estruturas tridimensionais usuais de edifícios altos.* 1983. Dissertação (Mestrado), EESC-USP.
11. RIOS, B.M.C. *Análise tridimensional e envoltória de esforços em edifícios altos sujeitos a ações verticais e laterais.* 1991. Dissertação (Mestrado), EESC-USP.
12. SAVASSI, W. *Introdução ao método dos elementos finitos em análise linear de estruturas.* 1996. São Carlos, Editora da EESC-USP.
13. TURNER, M.J.; CLOUGH, R.W.; MARTIN, H.C.; TOPP, L.J. "Stiffness and deflection analysis of complex structures." *J. Aero. Sci.*, 1956.
14. VLASSOV, B.Z. *Pièces longues en voiles minces.* Paris, Eyrolles, 1962.
15. ZIENKIEWICZ, O.C. *El método de los elementos finitos en la mecánica de estructuras y del continuo.* Editorial Reverté, Barcelona, 1967.
16. WARBURTON, G.B. *The Dynamical Behaviour of Structures.* Oxford, Pergamon Press, 1976.
17. RINALD, R. *Turbo Pascal 7.0 — Comandos e Funções.* 3rd ed., São Paulo, Editora Érica, 1993.
18. WOOD, S. *Guia do Usuário.* São Paulo, McGraw-Hill, 1987.

Annexes — where to find them in this repository

Original annex (PDF pages)	In this repository
List of DLEARN variables (1–16)	`../dlearn-input-format.md` — re-derived from the Fortran; note the corrections listed there (IHERM column and range, gravity card, `*end`, time-function columns)
PPD source program (17–75)	`../../ppd/legacy-pascal/PPD.PAS` and `GERAL.PAS`, byte-identical to the 2001 files
Input data file (76)	`../../examples/trelica/` — the same truss, with the control card and gravity card corrected; the 2001 file is on p. 76 of the PDF
Output data file (77–100)	`examples/trelica/reference/*.out` — outputs of the rebuilt executables (the 2001 output was produced with the mis-read control card and a gravity field of −10 000, hence its larger displacements)